



ARTIFICIAL INTELLIGENCE

OBJECT ORIENTED PROGRAMMING(JAVA)

Instructor: Muhammad Shahid Bhatti

Name: M. ARMAN HAMZA

Reg No.: SP26-BAI-034

Object-Oriented Analysis of a Smart Campus Allocation System

Task 1: Stakeholder and Requirement Analysis

1. Stakeholder Identification and Primary Needs

The Smart Campus Learning Space Allocation System involves diverse institutional stakeholders with distinct operational priorities:

- **Students:** Require fair, transparent access to study rooms and labs for group collaboration and personal study without procedural friction.
- **Faculty Members:** Require guaranteed priority booking for scheduled academic teaching, lab sessions, and urgent research activities.
- **Maintenance Staff:** Require authority to override confirmed reservations instantly to execute unscheduled hazard repairs and preventive facility maintenance.
- **Disability Support Staff:** Require strict enforcement of accessibility allocations so that specialized, accessible rooms remain available for students with physical needs without public exposure of sensitive medical data.

2. Genuine Requirement Conflicts and Architectural Decisions

To reconcile competing operational goals, three critical system conflicts were identified and resolved through explicit design decisions:

Requirement Conflict	Competing Stakeholders	Resolution / Design Decision
Fair Access vs. Faculty Priority	Students vs. Faculty	System establishes a layered allocation window: Faculty can reserve spaces up to 30 days in advance, while student recurring reservations open 7 days prior. Emergency faculty preemptions require administrative approval.
High Room Utilization vs. Instant Maintenance Blocks	Administration vs. Maintenance Staff	Maintenance overrides are given immediate priority to soft-lock spaces. The system

		automatically executes an automated re-allocation procedure for bumped bookings.
Accessible Room Allocation vs. Student Privacy (Integrated Condition)	Disability Support vs. Student Privacy	Accessibility attributes are tracked via encrypted system tokens. Peer users view room status as 'Reserved' without seeing disability-related booking flags.

3. Distinction Between Given Scenario Facts and Introduced Assumptions

Given Scenario Facts:

- Users encompass multiple roles (students, faculty, admin, security, maintenance, disability support).
- Room categories include classrooms, labs, seminar rooms, and project spaces with varying capacity and equipment.
- Emergency events force cancellation or relocation while maintaining audit traces.

Introduced System Assumptions:

- Fair access for students is defined as a cap of maximum 6 hours of advance bookings per week per individual.
- Accessible room features (e.g., ramps, specialized desks) are flagged as system constraints and auto-reserved for qualified role profiles prior to general release.
- Simultaneous booking requests occurring within a 500ms window are resolved via timestamp ordering and user role priority.

Task 2: Object-Oriented Conceptual Model

1. Proposed Conceptual Classes and Responsibilities

Class Name	Core Responsibility	Attributes & Operations
User	Represents system actors and maintains core user profile identity.	Attributes: userId, name, email Operations: getActiveRoleContext(),

		getPermissions()
RoleContext	Encapsulates dynamic operational permissions associated with user roles.	Attributes: roleType, permissionSet, priorityLevel Operations: validatePermission(), elevatedAccess()
Space	Models physical campus locations and their constraints.	Attributes: spaceId, capacity, isAccessible, currentStatus Operations: checkAvailability(), softBlock(), release()
ReservationRequest	Represents a booking application throughout its lifecycle.	Attributes: requestId, timeSlot, purpose, requestState Operations: submit(), updateState(), cancel()
ApprovalPolicy	Encapsulates conflict resolution rules and authorization logic.	Attributes: policyId, priorityMatrix, autoApproveThreshold Operations: evaluate(request), resolveConflict()
Equipment	Models scarce or movable hardware resources assigned to spaces.	Attributes: equipmentId, name, isMovable, status Operations: allocateToSpace(), detach()
ScheduleCatalog	Manages spatial-temporal availability indexes to avoid overlaps.	Attributes: catalogId, timeSlotMap Operations: queryAvailability(), lockSlot(), unlockSlot()
AuditEvent	Captures immutable records of state transitions and administrative overrides.	Attributes: logId, timestamp, actorId, actionType, details Operations: writeLog(), fetchAuditTrail()
NotificationService	Handles user communications during cancellations or	Attributes: serviceId, deliveryQueue

relocations.

Operations: dispatchAlert(),
notifyRelocation()

2. Conceptual UML Class Diagram

The class diagram below illustrates the relationships, multiplicities, and structural composition of the Smart Campus system:

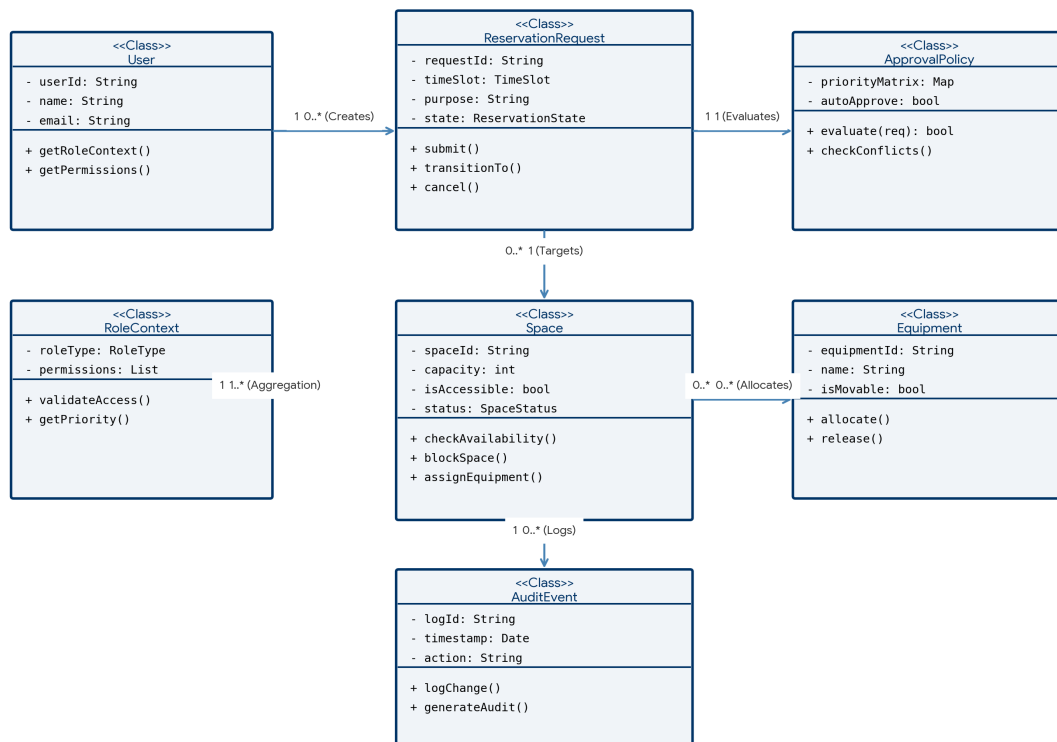


Figure 1: Conceptual UML Class Diagram for Smart Campus Allocation System

3. Responsibility Separation and Prevention of Monolithic Design

To prevent a single 'God Class' from controlling the system, responsibilities are strictly decentralized using Single Responsibility Principles (SRP):

- **User vs. RoleContext:** User manages core identity, while permission evaluation is delegated to dynamic RoleContext objects.
- **Space vs. ScheduleCatalog:** Space encapsulates physical characteristics (capacity, accessibility), whereas ScheduleCatalog handles time-slot indexing and conflict checking.
- **ReservationRequest vs. ApprovalPolicy:** ReservationRequest holds state data, but evaluation rules are completely isolated within ApprovalPolicy, allowing policies to change dynamically without modifying reservation entities.

Task 3: Abstraction, Encapsulation, and Invariants

1. Real-World Abstraction in Key Classes

Three core classes exemplify object-oriented abstraction by modeling essential system attributes while suppressing real-world complexity:

- **Space Abstraction:** Abstracts a physical room into operational constraints—seating capacity, accessibility compliance, and availability slots—omitting non-essential architectural details like wall color or structural materials.
- **ReservationRequest Abstraction:** Abstracts human scheduling interactions into a formal state machine containing time-slot entities, priority levels, and lifecycle states.
- **RoleContext Abstraction:** Abstracts complex institutional positions (e.g., Teaching Assistant, Researcher) into discrete sets of explicit permission flags and priority metrics.

2. Encapsulation and Controlled Data Access

Encapsulation enforces data hiding and prevents arbitrary modification of internal state:

Class	Hidden Data (Private)	Controlled Operations	Encapsulation Rationale
Space	currentStatus, maintenanceLogs	softBlockSpace(), markOperational()	Direct status modification is private. Maintenance status can only be toggled through validated operational methods.
ReservationRequest	requestState, approvalAuditTrace	transitionState(newState), attachAudit()	State attributes are hidden. State changes must pass through transition functions enforcing lifecycle rules.
User / RoleContext	accessibilityProfile, permissions	grantRole(), revokeRole()	Accessibility details are encapsulated within private tokens to preserve student privacy under the

integrated condition.

3. System Invariants and Protection Rationale

- **Invariant 1 (Overbooking Prevention):** No two confirmed ReservationRequests can overlap in timeSlot for the same Space instance.
- **Invariant 2 (Accessibility Priority):** A Space flagged as isAccessible=true cannot be allocated to a standard reservation if an unfulfilled request requiring accessibility exists for that time slot.
- **Invariant 3 (Maintenance Supremacy):** If Space.status == MAINTENANCE, no ReservationRequest targeting that Space can enter the CONFIRMED state.
- **Invariant 4 (Audit Traceability):** Every state change in ReservationRequest must instantly create an immutable AuditEvent entry before completing.

Role of Encapsulation: Encapsulation prevents invalid states by making state variables private and mandating that all modifications pass through validation methods. For example, if a user attempts to confirm a booking, transitionState() evaluates Invariant 1 and Invariant 3 before updating requestState. This prevents direct, illegal state alterations.

Task 4: Alternative Designs and Decision

1. Design Representation Comparison

To handle users holding dynamic or multiple roles (e.g., a student acting as a Teaching Assistant), two architectural approaches were evaluated:

- Option A (Inheritance Hierarchies): User is an abstract base class extended by StudentUser, FacultyUser, and TAUser subclasses.
- Option B (Composition with Role Objects / Strategy Pattern): User maintains a dynamic collection of RoleContext objects.

Evaluation Criterion	Option A: Class Inheritance	Option B: Composition (Role Objects)	Analysis / Impact
Multiple Roles Handling	Rigid. Requires combinatorial class explosion (e.g., StudentTAFaculty).	Seamless. User holds a List<RoleContext> instance.	Composition easily scales as roles multiply.
Runtime Role Switching	Impossible without instantiating new objects and remapping	Dynamic. Roles can be added or deactivated at	Composition supports mid-semester

	references.	runtime.	permission updates.
Coupling & Duplicate Behavior	High coupling. Behavior duplicated across hybrid subclasses.	Low coupling. Permissions isolated within discrete role strategies.	Composition eliminates code duplication.

2. Design Decision and Requirement Justification

Chosen Design: Option B (Composition using Role Objects).

Justification against Requirements: COMSATS campus rules require that a person holding multiple roles (e.g., a Student who becomes a TA) must experience seamless, dynamic permission shifts during the semester. Inheritance creates rigid class coupling and fails when an individual acts in dual capacities simultaneously. Composition allows the system to evaluate permissions dynamically based on active RoleContext strategies, satisfying institutional requirements without modifying user entity references.

Task 5: Behavior and Edge Cases

1. Reservation Lifecycle State Transitions

Current State	Trigger Event	Next State	Enforcing Object & Action
SUBMITTED	ApprovalPolicy.evaluate() == PASS	CONFIRMED	ApprovalPolicy validates slot availability and role priority.
CONFIRMED	Maintenance override issued	BLOCKED / CANCELLED	Space invokes softBlock(), triggering auto-relocation or cancellation.
CONFIRMED	Emergency campus closure	CANCELLED / RELOCATED	NotificationService dispatches alerts; AuditEvent logs emergency displacement.

SUBMITTED	ApprovalPolicy.evaluate() == FAIL	REJECTED	ApprovalPolicy records conflict logs in AuditEvent.
CONFIRMED	User releases booking	COMPLETED	ScheduleCatalog releases locked time slot.

2. Resolution of System Edge Cases

- Case 1: Two Simultaneous Requests for the Same Space**
 Enforcing Object: ScheduleCatalog (using synchronized isolation).
 Resolution: The catalog processes requests via atomic database operations. The request with the higher priority RoleContext locks the slot first. The second request is rejected or placed in a waiting queue.
- Case 2: Unscheduled Maintenance Block After Confirmation**
 Enforcing Object: Space and NotificationService.
 Resolution: Space status updates to MAINTENANCE, revoking existing confirmations. The system searches for an equivalent available space (matching capacity and accessibility rules) and auto-relocates the booking while dispatching alerts.
- Case 3: Emergency Cancellation and Relocation**
 Enforcing Object: SystemAdmin / NotificationService.
 Resolution: Triggers mass cancellation across affected rooms, logs emergency overrides in AuditEvent for accountability, and dispatches SMS/email notifications.

Task 6: Independent Exploration

1. Concept Definition: Role-Based Access Control (RBAC) & Dynamic Security Contexts

Role-Based Access Control (RBAC) is a security abstraction paradigm wherein system permissions are associated with specific institutional roles rather than individual user identities. Instead of assigning authorization rules directly to users, rights are encapsulated within role definitions. Users are subsequently assigned one or more roles, dynamically inheriting the associated access rights (Ferraiolo et al., 2001; Sandhu et al., 1996).

Application to System Model: In our Smart Campus model, integrating RBAC via the RoleContext class significantly enhances security flexibility. When a user requests a space, the ApprovalPolicy queries the active RoleContext strategy rather than evaluating raw user properties. For example, when a student operates as a Teaching Assistant, their User instance temporarily activates the TA RoleContext, granting

elevated room booking privileges without hardcoding role checks into core domain objects. This decoupled design ensures smooth security auditing and seamless permission management.

REQUIRED GENERATIVE AI DISCLOSURE

Generative AI tools (ChatGPT/Gemini) were utilized to brainstorm architectural edge cases, refine formal invariant descriptions, and draft initial UML structural layouts. All domain modeling, class responsibility allocations, trade-off evaluations, and structural design decisions were independently verified, adapted, and justified against COMSATS campus requirements.